ABB OT200E03 Switch Disconnector — AR/MR Assembly Training System

Developer Manual and Technical Reference

Version: 1.0 — August 2026
Platform: Meta Quest 3 (Mixed Reality) + Android Mobile (Augmented Reality)
Engine: Unity 6 (URP)
Repository: ABB-Assembly-MR-Demo

Table of Contents

---

1. Project Overview

This application is a dual-platform industrial XR assembly training tool built for the ABB OT200E03 200A three-pole switch disconnector (IEC 60947-3). It allows factory workers to practise disassembling and reassembling all 45 components of the physical switch in an augmented or mixed reality environment before working on the actual hardware.

The application runs on two platforms from a single Unity project:

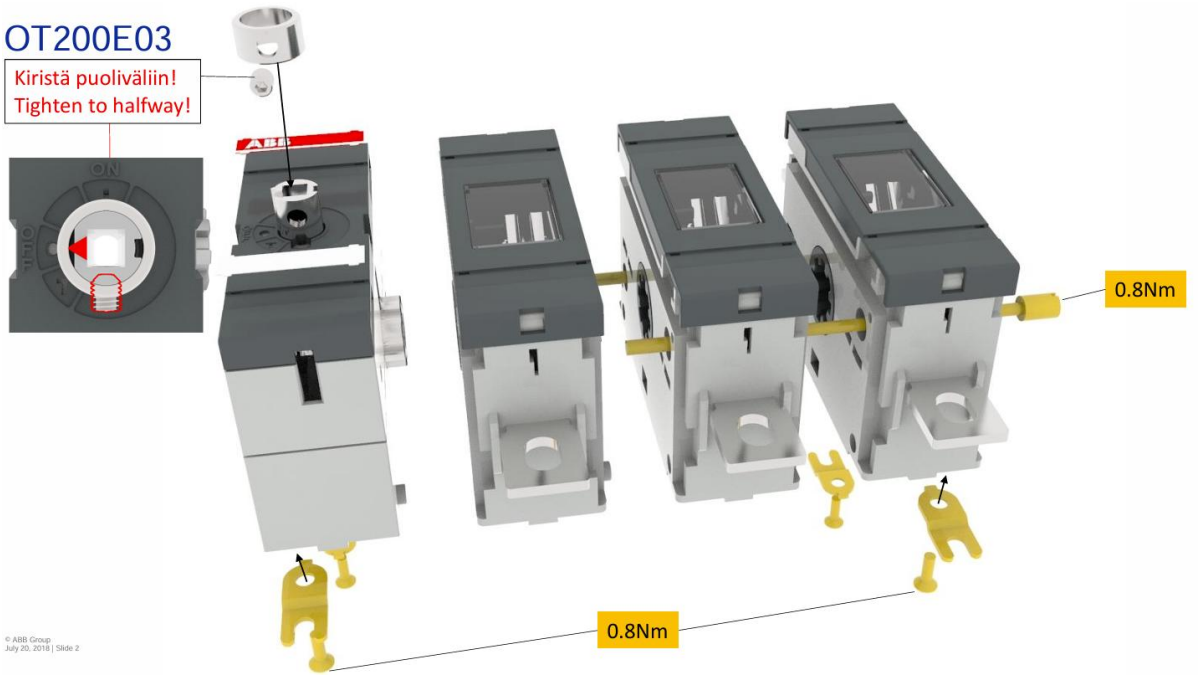
Mixed Reality (Meta Quest 3): The user wears the headset and sees the real world through passthrough cameras. Virtual switch components are spawned inside the highlighted areas on a detected real surface, i.e., a table or floor, when the grip button of the right controller is pressed. The user picks up parts using the trigger button of the controllers and assembles them in a fixed virtual assembly zone visible through a ghost mesh overlay system.

Augmented Reality (Android Mobile): The user holds their phone and points the back camera at a flat surface. Parts are spawned inside the highlighted areas on a detected real surface. Parts can be picked and placed using the touchscreen.

Both platforms share all assembly logic, prerequisite rules, visual guide system, physics configuration, hint system, and part label system. Only the input handling and platform-specific XR session management differ between scenes.

The switch has 45 physical components organised into five groups:

- **Driver Module**: the primary housing containing the operating shaft, interlocking nuts, internal fasteners, top cover, red and white strips, shaft collar, clamping bolt, external fasteners, and mounting clips (18 parts)
- **Receiver Module 1**: right and left housing shells, interlocking drive ring, contact cover, contact window, and two terminal lugs (7 parts)
- **Receiver Module 2**: identical structure to Receiver Module 1 (7 parts)
- **Receiver Module 3**: identical structure to Receiver Modules 1 and 2, plus external fasteners and mounting clips (11 parts)
- **Modular Rods**: two long rods that thread through all four modules and fasten the complete assembly (2 parts)



---

2. Repository Structure

```
ABB-Assembly-MR-Demo/
│
├── Assets/
│   ├── Editor/
│   │   ├── AssemblyManagerSetup.cs
│   │   ├── SnapZoneGhostSetup.cs
│   │   ├── SnapZonePositioner.cs
│   │   └── VisualGuideGenerator.cs
│   │
│   ├── Materials/
│   │   ├── Contact Windows.mat
│   │   ├── Fasteners and Clips.mat
│   │   ├── Housings.mat
│   │   ├── Lugs and Rings.mat
│   │   ├── Red Strip.mat
│   │   ├── SnapZoneGhostMaterial.mat
│   │   ├── SwitchPartPhysics.physicsMaterial
│   │   └── Top Covers.mat
│   │
│   ├── MobileARTemplateAssets/
│   │   ├── Prefabs/
│   │   │   └── Switch Parts/ (Used in SwitchSpawnLayout-AR.asset)
│   │   │       ├── Driver-Module/ (18 part prefabs for AR-SampleScene)
│   │   │       ├── Receiver-Module-1/ (7 part prefabs for AR-SampleScene)
│   │   │       ├── Receiver-Module-2/ (7 part prefabs for AR-SampleScene)
│   │   │       ├── Receiver-Module-3/ (11 part prefabs for AR-SampleScene)
│   │   │       ├── Module-Rod-A (for AR-SampleScene)
│   │   │       └── Module-Rod-B (for AR-SampleScene)
│   │   └── (AR Mobile template default assets)
│   │
│   ├── MRTemplateAssets/
│   │   ├── Prefabs/
│   │   │   └── Switch Parts/ (Used in SwitchSpawnLayout-MR.asset)
```

```
│   │   │       ├── Driver-Module/ (18 part prefabs for MR-SampleScene)
│   │   │       ├── Receiver-Module-1/ (7 part prefabs for MR-SampleScene)
│   │   │       ├── Receiver-Module-2/ (7 part prefabs for MR-SampleScene)
│   │   │       ├── Receiver-Module-3/ (11 part prefabs for MR-SampleScene)
│   │   │       ├── Module-Rod-A (for MR-SampleScene)
│   │   │       └── Module-Rod-B (for MR-SampleScene)
│   │   └── (MR template default assets)
│   │
│   ├── Objects/
│   │   │   │       ├── 1-Original ABB Model/ (The STP file of ABB Switch model provided
by ABB)
│   │   │   │       ├── 2a-SolidWorks Models (Coarse)/ (The STL files converted from
the STP file in SolidWorks with settings: coarse, binary)
│   │   │   │       ├── 2b-SolidWorks Models (Fine)/ (The STL files converted from the
STP file in SolidWorks with settings: fine, binary)
│   │   │   ├── ABB_Switch_Coarse_Transformed (Fixed coordinate mismatch CAD import
from Blender (scale 1,1,1, rotation 0,0,0) – Used Coarse SolidWorks STL files,
should be used to create ABB Switch prefab for AR Mobile app)
│   │   │   ├── ABB_Switch_Fine_Transformed (Fixed coordinate mismatch CAD import from
Blender (scale 1,1,1, rotation 0,0,0) – Used Fine SolidWorks STL files, should be
used to create ABB Switch prefab for MR application)
│   │   │   ├── ABB_Switch_Fine_Untransformed.fbx (CAD model imported from Blender which
had the coordinate mismatch issue (scale 100,100,100, rotation -89.98,0,0) and was
used to create ABB-Switch-Original.prefab)
│   │   │   ├── Ding.wav   (parts' snapping sound)
│   │   │   ├── SwitchSpawnLayout-AR.asset (0.08 offset to fit on a mobile screen)
│   │   │   └── SwitchSpawnLayout-MR.asset (0.25 offset to fit on the Quest display)
│   │   │
│   ├── Prefabs/
│   │   │   ├── ABB-Switch-Original.prefab (The 3D Unity model with coordinate mismatch
issue, created using ABB_Switch_Fine_Untransformed.fbx, scale: 0.002)
│   │   │   ├── ABB_Switch_AR.prefab (Unpacked from ABB-Switch-Original.prefab, 45 parts
scaled to 0.1)
│   │   │   ├── ABB_Switch_MR.prefab (Unpacked from ABB-Switch-Original.prefab, 45 parts
scaled to 0.2)
│   │   │   ├── AssemblyHints.prefab
│   │   │   ├── PartLabel.prefab
│   │   │   ├── SpawnPreviewPrefab-AR.prefab (material: SnapZoneGhostMaterial.mat)
│   │   │   └── SpawnPreviewPrefab-MR.prefab (material: Contact Windows.mat)
│   │   │
│   ├── Scenes/
│   │   │   ├── MR-SampleScene.unity
│   │   │   └── AR-SampleScene.unity
│   │   │
│   ├── Scripts/
│   │   │   ├── AR Scripts/
│   │   │   │   └── ARAdaptiveUI.cs
│   │   │   │   └── LostRecoveryPartAR.cs
│   │   │   │   └── SwitchSpawnerAR.cs
│   │   │   ├── MR Scripts/
│   │   │   │   └── SwitchSpawnerMR.cs
│   │   │   ├── ARPlaneColliderGenerator.cs
│   │   │   ├── AssemblyHintSystem.cs
│   │   │   ├── AssemblyManager.cs
│   │   │   ├── AssemblySnapZone.cs
│   │   │   ├── PartLabelDisplay.cs
│   │   │   ├── PartTypeSelectFilter.cs
│   │   │   ├── SpawnPreview.cs
│   │   │   ├── SwitchPart.cs
```

```
│   │   ├── SwitchPartType.cs
│   │   ├── SwitchSpawnLayout.cs
│   │   └── VisualSnapGuide.cs
│   │
│   └── Samples/ (XRI Starter Assets, auto-generated)
│
├── Packages/
│   └── manifest.json
│
└── ProjectSettings/
    ├── ProjectSettings.asset
    └── (all other project settings)
```

---

3. Setting Up the Project from GitHub

Prerequisites

Install these before cloning:

- **Unity Hub** (latest)
- **Unity 6000.3.x LTS** with Android Build Support (including Android SDK and NDK)
- **Meta Horizon Link** (for Quest 3 testing via PC streaming)
- **Android USB drivers** (for direct device build)
- **Git LFS** — this project uses Git Large File Storage for FBX and prefab assets. Run git lfs install before cloning.

Clone and Open

bash

git lfs install

git clone https://github.com/FaizanTanwir/ABB-Assembly-MR-Demo.git

Open Unity Hub → Add → select the cloned folder. Unity imports all packages automatically from Packages/manifest.json. The first import takes approximately 10–15 minutes.

Required Packages (auto-installed from manifest)

Package	Purpose
com.unity.xr.arfoundation	AR Foundation (planes, anchors, camera)
com.unity.xr.arcore	ARCore for Android phones
com.unity.xr.openxr	OpenXR cross-platform XR standard

Package	Purpose
com.unity.xr.interaction.toolkit	XR grab, socket, ray interactors
com.unity.xr.management	XR plugin management
com.unity.inputsystem	New Input System (touch, controller)
com.unity.textmeshpro	TMP for labels and hints

Post-Clone Steps (Required Before Building)

After Unity finishes importing, three manual steps are required because the allZones list in AssemblyManager is a serialised Inspector reference that cannot be stored in code:

Step 1: Open MR-SampleScene. In the menu bar, click **ABB Tools → Populate AssemblyManager Zone List**. This finds all 45 AssemblySnapZone components and fills the AssemblyManager's allZones list. Without this, no snap zones are active.

Step 2: With the ABB_Switch reference GameObject enabled in the hierarchy (it is disabled by default — temporarily enable it), click **ABB Tools → Generate Visual Snap Guides (Name-Based)**. This creates the VisualGuides GameObject with 45 ghost meshes at the correct part positions—re-disable ABB_Switch after generation.

Step 3: Click **ABB Tools → Align Zone Triggers to Visual Guides**. This maps each snap zone's Box Collider center and size to its corresponding visual guide position, ensuring triggers fire where the user sees the ghost mesh.

Repeat Steps 1–3 for AR-SampleScene.

---

4. Plugin and Player Settings

XR Plug-in Management — Android Tab

Navigate to **Edit → Project Settings → XR Plug-in Management → Android tab (the robot icon)**.

For Meta Quest 3 Build

Setting	Value
Google ARCore	✗ Unchecked
OpenXR	✓ Checked
Meta Quest feature group	✓ Checked

Setting	Value
Android XR feature group	✗ Unchecked (mutually exclusive with Meta Quest)

Under **OpenXR → Enabled Interaction Profiles**:

- Meta Quest Touch Plus Controller Profile ✓
- Oculus Touch Controller Profile ✓

Under **OpenXR → Meta Quest feature group**, enable:

- Meta Quest Support ✓
- Meta Quest: Camera (Passthrough) ✓
- Meta Quest: Planes ✓
- Meta Quest: Anchors ✓
- Meta Quest: Session ✓
- Meta Quest: Raycasts ✓
- Meta Quest: Bounding Boxes ✓
- Meta Quest: Boundary Visibility ✓
- Meta Quest: Occlusion ✓
- Hand Tracking Subsystem ✓
- Meta Hand Tracking Aim ✓
- Composition Layers Support ✓

For Android Mobile (ARCore) Build

Setting	Value
Google ARCore	✓ Checked
OpenXR	✗ Unchecked

No additional feature groups needed. ARCore handles all AR subsystem initialisation automatically.

XR Plug-in Management — Desktop Tab

Setting	Value
XR Simulation	✓ Checked (for editor testing of AR scenes)

Setting	Value
Initialise XR on Startup	✓ Checked

Player Settings — Android Tab

Navigate to **Edit → Project Settings → Player → Android tab**.

Setting	Value	Reason
Minimum API Level	Android 12L (API 32)	Meta Quest minimum requirement
Target API Level	Android 16 (API 36)	To match my Android test device
Scripting Backend	IL2CPP	Required for Android XR builds
Target Architectures	ARM64 ✓	Quest and modern Android phones
Auto Graphics API	✗ Unchecked	
Graphics APIs	Vulkan (primary)	Best performance on Quest 3
Internet Access	Auto	
Active Input Handling	Input System Package (New)	Required for XRI and touch

OpenXR Render Settings

Navigate to **Project Settings → XR Plug-in Management → OpenXR**:

Setting	Value
Render Mode	Single Pass Instanced \ Multi-view
Latency Optimization	Prioritise Input Polling
Depth Submission Mode	None
Foveated Rendering Api	Legacy

---

5. Unity Template Default GameObjects Explained

MR Template Default Objects (MR-SampleScene)

These GameObjects came pre-configured with the MR Template and are interconnected through serialised Unity Event references. Do not delete them — many are required even when their visual output is disabled.

MR Interaction Setup

The root coordinator of the entire MR experience. Contains:

- **Input Action Manager**: loads the XRI Default Input Actions asset into Unity's Input System so all controller button bindings are active
- **XR Interaction Manager**: the central singleton that coordinates all XR interactions. Every XR interactor (ray, socket) and interactable (grab) registers with this manager. It resolves which interactor is selecting which interactable and manages state transitions
- **EventSystem**: Unity's UI event routing. Required for any Canvas button to receive click events from the controller ray

Inside its XR Origin child:

- **XR Origin component**: defines the relationship between real-world tracking space and Unity world space. The Camera Offset child moves with head tracking
- **AR Session**: manages the lifecycle of the AR experience — starts ARCore/OpenXR AR subsystems, handles permissions, manages session state (running, paused, stopped). Only one AR Session may exist in a scene
- **AR Plane Manager**: subscribes to the platform's plane detection system and creates plane visualisation GameObjects (using the Plane Prefab) for each detected flat surface. In this project, the Plane Prefab reference has been set to None (no visual planes) while ARPlaneColliderGenerator adds physics MeshColliders separately
- **AR Bounding Box Manager**: detects 3D bounding boxes around real-world objects. Bounding Box Prefab is set to None to suppress visuals while keeping detection active
- **AR Anchor Manager**: provides persistent spatial anchors. Used by the MR template's anchor system. Not directly used by the assembly logic but required for full scene understanding functionality
- **AR Raycast Manager**: enables AR Foundation plane-based raycasting. Used by SwitchSpawnerMR to find where the user is pointing when spawning parts
- **AR Camera Manager** (on Main Camera child): activates the passthrough camera feed. This component is disabled by default and enabled at runtime by the AR Feature Controller through the UI prefab's Boolean Toggle
- **AR Feature Controller** (on XR Origin): the custom MR template script that connects UI toggles to AR subsystem states. Holds serialised references to the AR Camera Manager, AR Plane Manager, AR Bounding Box Manager, and the Fade

Material in the Environment prefab. Its Unity Events fire when the coaching UI completes, enabling passthrough. Without the UI and Environment GameObjects in the hierarchy, these references are null and passthrough never activates — this was the root cause of the passthrough problem documented during development

- **Occlusion Manager**: controls hand occlusion (real hands occluding virtual objects). Holds references to Quest Settings and Android XR Settings ScriptableObjects
- **Near-Far Interactor** (on Left/Right Controller children): the controller's primary interaction component in XRI 3.x. Replaces the older separate XR Direct Interactor and XR Ray Interactor. Casts a ray from the controller position for far interactions and detects direct contact for near interactions. The physics layer mask on this component must include the SwitchPart and SwitchPartGrabbed layers — if not, the controller ray passes through switch parts

Goal Manager

Controls the step-by-step coaching tutorial at the start. Drives the Coaching UI panels, enables/disables passthrough through the Passthrough Toggle reference, and controls the AR Feature Controller through Unity Events. In this project, the Goal Manager is present in the hierarchy, but its coaching UI children can be disabled. The Goal Manager's goal-complete event was considered for triggering the spawn guard, but the simpler grip-button-only guard approach was used instead.

Object Spawner

The MR template's built-in object spawning system. Uses an ARInteractionSpawnTrigger to spawn prefabs when the controller ray hits a detected plane. Disabled in this project because SwitchSpawnerMR handles all spawning. If re-enabled, it would conflict with SwitchSpawnerMR's spawn logic.

UI

Contains the MR template's complete hand menu, coaching UI panels, spatial panels, and Boolean Toggles. Although most children are disabled visually, the root UI GameObject must remain active because the AR Feature Controller holds serialised references to Boolean Toggle objects inside it. Disabling the root UI disables those Toggle components, which breaks the AR Feature Controller's passthrough activation event chain, resulting in a black screen.

Environment

Contains the skybox fade material referenced by the AR Feature Controller. The FadeMaterial.FadeSkybox method is called by the AR Feature Controller when passthrough activates, fading out the virtual skybox to reveal the real camera feed.

Environment's visual mesh (the room geometry) can be disabled, but the GameObject must remain active for its fade material to be reachable.

Lighting

Directional Light for scene illumination. Kept at default settings.

AR Mobile Template Default Objects (AR-SampleScene)

XR Origin (AR Rig)

Mobile AR equivalent of the MR template's XR Origin. Simpler structure because there are no controllers and no hand tracking.

- **Tracked Pose Driver** (on XR Origin): reads the phone's IMU and ARCore visual odometry and applies the result to Camera Offset every frame. This keeps virtual objects anchored to real-world positions as the phone moves
- **AR Camera Manager** (on Main Camera): activates the phone's back camera feed through ARCore
- **AR Camera Background** (on Main Camera): renders the phone camera texture as the scene background behind all 3D content. This component is specific to mobile AR and does not exist in the MR template. On mobile, the real world is shown as a camera texture rendered by AR Camera Background. On Quest, the real world is shown through OpenXR's compositor passthrough layer, which is a fundamentally different mechanism
- **Screen Space Ray Interactor** (child of Camera Offset): converts screen touch positions into 3D rays for XRI interaction. Contains:
    - **Touchscreen Gesture Input Loader**: registers touch gesture input bindings with Unity's Input System
    - **Screen Space Ray Pose Driver**: converts 2D touch position into a 3D ray origin and direction using Camera.ScreenPointToRay
    - **XR Ray Interactor**: the same component used on Quest controllers. Works identically whether driven by a controller or a screen-space ray
    - **XR Interaction Group**: manages priority when multiple interactions are possible
    - **Touchscreen Hover Filter**: prevents hover false-positives when no finger is touching
    - **Select/Rotate/Scale Input children**: map single-tap to select, two-finger twist to rotate, two-finger pinch to scale (not used by the assembly app but present from the template)

AR Session (standalone)

Same function as in MR template. On mobile, ARCore handles camera permission automatically through this component — no custom Permissions Manager is needed, unlike the Quest, which requires the explicit USE_SCENE permission request through Permissions Manager Variant.

Object Spawner (Camera Offset child)

Disabled. Same role as in MR template — replaced by SwitchSpawnerAR.

UI

Screen-space Canvas with coaching UI and options modal. Disabled in AR-SampleScene. Unlike the MR template UI, the AR mobile UI is not referenced by any AR feature controller — disabling it does not affect passthrough, which is handled entirely by AR Camera Background.

---

6. Scene Hierarchies

MR-SampleScene Hierarchy

```
MR-SampleScene
│
├── MR Interaction Setup   [MR template default – keep active]
│    ├── Input Action Manager
│    ├── XR Interaction Manager
│    ├── EventSystem
│    ├── Goal Manager
│    ├── AR Session
│    ├── Object Spawner   [disabled – SwitchSpawnerMR used instead]
│    └── XR Origin (XR Rig)
│         ├── Camera Offset
│         │    ├── Main Camera   [AR Camera Manager, Universal Camera Data]
│         │    ├── Gaze Interactor
│         │    ├── Right Controller
│         │    │    └── Near-Far Interactor  [SwitchPart + SwitchPartGrabbed layers in
raycast mask in filtering settings of Sphere and Curve Interaction Caster]
│         │    ├── Left Controller
│         │    │    └── Near-Far Interactor  [SwitchPart + SwitchPartGrabbed layers in
raycast mask in filtering settings of Sphere and Curve Interaction Caster]
│         │    ├── Left Hand
│         │    ├── Right Hand
│         │    └── Hand Visualizer
│         ├── AR Plane Manager     [Prefab: AR Plane]
│         ├── AR Bounding Box Manager  [Prefab: None, Default: AR Bounding Box]
│         ├── AR Anchor Manager
│         ├── AR Raycast Manager
│         ├── ARPlaneColliderGenerator.cs  [adds MeshColliders to detected planes at
runtime]
│         ├── Occlusion Manager
│         ├── Locomotion
│         └── Hands Smoothing Post Processor
```

```
│
├── Permissions Manager
├── UI                                    [root active]
│   ├── Coaching UI
│   ├── Spatial Panel Manipulator
│   ├── Tap Tooltip
│   └── Hand Menu Setup MR Template Variant
├── Environment           [root active]
├── Lighting
│
├── AssemblyManager        [empty GO]
│   ├── AssemblyManager.cs (Snap Zones, filled by 'Populate Zone List' in ABB Tools)
│   ├── AssemblyHintSystem.cs
│   └── PartLabelDisplay.cs (Prefab: Part Label)
│
├── SwitchSpawner                         [empty GO]
│   └── SwitchSpawnerMR.cs
│       Fields: Layout=SwitchSpawnLayout-MR,
│               Snap Zones Root=SnapZones-Transformed,
│               Visual Guides Root=VisualGuides,
│               Spawn Action=XRI RightHand/Select
│
├── SpawnPreview                          [empty GO]
│   └── SpawnPreview.cs
│       Fields: Preview Prefab=SpawnPreview-MR
│
├── VisualGuides         [generated by ABB Tools; hidden until spawn]
│   ├── Guide_BottomHousing_0
│   ├── Guide_TopHousing_0
│   ├── Guide_OperatingShaft_0
│   └── ... (42 more Guide_ GameObjects)
│
├── SnapZones-Transformed   [all zones at local 0,0,0; invisible triggers]
│   ├── DriverModule_Zones
│   │   ├── BottomHousing_0  [Box Collider trigger, Mesh Renderer (Snap Zone Ghost
Material), XRSocketInteractor, PartTypeSelectFilter, AssemblySnapZone (script)]
│   │   │   └── AssemblySnapZone.cs
│   │   │       Fields: zoneId=BottomHousing_0
│   │   │       Accepted Part Type = Bottom Housing
│   │   │       Accepted Module Index = 0
│   │   │       Ignore Module Index = unchecked (can be checked)
│   │   │       Visual Guide = Guide_BottomHousing_0
│   │   │       AND prereq list = empty
│   │   │       OR prereq list = empty
│   │   │       Audio (Snap Sound) = Ding.wav
│   │   ├── InterlockingNut_0_A [same components; AND prereq: BottomHousing_0]
│   │   ├── InterlockingNut_0_B  [prereq: BottomHousing_0]
│   │   ├── InterlockingNut_0_C  [prereq: BottomHousing_0]
│   │   ├── InterlockingNut_0_D  [prereq: BottomHousing_0]
│   │   ├── ExternalFastener_0_Left [prereq: MountingClip_0_Left]
│   │   │   └── MountingClip_0_Left [prereq: BottomHousing_0]
│   │   └── ExternalFastener_0_Right [prereq: MountingClip_0_Right]
│   │       └── MountingClip_0_Right [prereq: BottomHousing_0]
│   ├── OperatingShaft_0     [prereq: BottomHousing_0]
│   ├── TopHousing_0   [prereq: BottomHousing_0, OperatingShaft_0, all 4 nuts]
│   ├── InternalFastener_0_Left   [prereq: TopHousing_0]
│   ├── InternalFastener_0_Right  [prereq: TopHousing_0]
│   ├── TopCover_0 [prereq: InternalFastener_0_Left, InternalFastener_0_Right]
│   │   ├── RedStrip_0            [prereq: TopCover_0]
```

```
│   │   │   └── WhiteStrip_0          [prereq: TopCover_0]
│   │   ├── ShaftReceiverCollar_0     [prereq: TopCover_0]
│   │   └── ClampingBolt_0            [prereq: ShaftReceiverCollar_0]
│   ├── ReceiverModule1_Zones
│   │   ├── RightHousing_1               [no prerequisites — phase 0]
│   │   │   ├── InterlockingDriveRing_1  [prereq: RightHousing_1]
│   │   │   ├── TerminalLug_1_Left       [prereq: RightHousing_1]
│   │   │   └── TerminalLug_1_Right      [prereq: RightHousing_1]
│   │   ├── LeftHousing_1         [prereq: RightHousing_1, DriveRing_1, both lugs]
│   │   └── ContactCover_1        [prereq: RightHousing_1, LeftHousing_1]
│   │       └── ContactWindow_1   [prereq: ContactCover_1]
│   ├── ReceiverModule2_Zones     [identical structure to Module1]
│   ├── ReceiverModule3_Zones     [same + ExternalFasteners and MountingClips]
│   └── ModularRod_Zones
│       ├── ModularRod_A [prereq: BottomHousing_0, all 6 receiver housings and nuts]
│       └── ModularRod_B [prereq: BottomHousing_0, all 6 receiver housings and nuts]
│
├── SnapZones-Reference         [original snap zones at mesh centers; disabled]
│
├── ABB_Switch [Prefab: ABB_Switch_MR, assembly reference; disabled during gameplay]
│   ├── Driver-Module
│   │   ├── Bottom-Housing
│   │   ├── Top-Housing
│   │   └── ... (16 more parts)
│   ├── Receiver-Module-1
│   ├── Receiver-Module-2
│   ├── Receiver-Module-3
│   ├── Modular-Rod-A
│   └── Modular-Rod-B
│
└── AssemblyHints                       [Screen Space — Camera Canvas]
    ├── hintText                        [TMP: "Place next (gold): PartType"]
    ├── countText                       [TMP: "X/45 parts (XX%)"]
    └── button
        └── RecoverButton     [OnClick: AssemblyHintSystem.RecoverLostParts]
```

AR-SampleScene Hierarchy

```
AR-SampleScene
│
├── AR Session                          [AR lifecycle, AR Input Manager]
├── Directional Light
├── EventSystem
├── UI              [disabled — clicking on AR mobile UI buttons spawns the parts]
├── XR Origin (AR Rig) [AR Plane and Raycast Manager, AR Plane Collider Generator]
│   └── Camera Offset
│       ├── Main Camera                 [AR Camera Manager, AR Camera Background]
│       ├── Object Spawner              [disabled]
│       └── Screen Space Ray Interactor  [touch → XRI ray]
│           └── Select Input
│           └── Rotate Input
│           └── Scale Distance Delta Input
│
├── SwitchSpawner                       [empty GO]
│   └── SwitchSpawner.cs (SwitchSpawnerAR)
│       Fields: Layout=SwitchSpawnLayout-AR,
│               Snap Zones Root=SnapZones-Transformed,
│               Visual Guides Root=VisualGuides
│
├── AssemblyManager                     [same scripts as MR-SampleScene]
```

```
│    ├── AssemblyManager.cs
│    ├── AssemblyHintSystem.cs
│    ├── LostPartRecoverAR.cs (absent in the MR-SampleScene)
│    └── PartLabelDisplay.cs
│
├── SpawnPreview
├── VisualGuides                          [same structure as MR-SampleScene]
├── SnapZones-Transformed                 [same structure as MR-SampleScene]
├── SnapZones-Reference                   [disabled]
├── ABB_Switch                            [disabled reference]
└── AssemblyHints                         [Screen Space – Camera Canvas]
```

---

7. All Scripts Reference

General Scripts (Assets/Scripts)

SwitchPartType.cs

Type: Enum (no MonoBehaviour)

Purpose: Defines all unique 19 part type identifiers used throughout the assembly system.

BottomHousing, TopHousing, InternalFastener, OperatingShaft,

ShaftReceiverCollar, ClampingBolt, RedStrip, WhiteStrip,

ExternalFastener, MountingClip, InterlockingNut, TopCover,

RightHousing, LeftHousing, InterlockingDriveRing, ContactCover,

ContactWindow, TerminalLug, ModularRod

Every SwitchPart component on a part prefab has a partType field of this enum type. Every AssemblySnapZone has an acceptedPartType field. The PartTypeSelectFilter uses this type to block the XR Socket Interactor from physically accepting wrong-type parts.

SwitchPart.cs

Type: MonoBehaviour

Attached to: all 45 part prefabs

Purpose: Identifies a part's type and module, manages visual highlighting (blue for valid hover, red for invalid, gold for hint, restored to original on reset), and notifies PartLabelDisplay when the controller ray hovers over the part.

Key fields:

- partType: SwitchPartType enum value set in the Inspector for each prefab
- moduleIndex: integer set by SwitchSpawnerMR/AR at instantiation time from the layout entry (0=Driver, 1/2/3=Receiver, -1=Global). Overrides the prefab's default value

How highlighting works: On Awake, the script reads the _BaseColor property from every child MeshRenderer's shared material and stores the original colours. All highlight methods use MaterialPropertyBlock to override _BaseColor without creating new material instances (GPU-efficient). On reset, stored original colours are restored.

Layer switching: When the XRGrabInteractable fires selectEntered (user grabs the part), the script moves the entire GameObject hierarchy to the SwitchPartGrabbed physics layer. On selectExited (released), it moves back to SwitchPart. The Physics Layer Collision Matrix is configured so SwitchPart and SwitchPartGrabbed do not collide — grabbed parts pass through resting parts without physics explosions.

SwitchSpawnLayout.cs

Type: ScriptableObject

Purpose: Data container for the spawn layout. Stored as an asset (.asset file). Not a MonoBehaviour — it holds serializable data that the spawner reads at runtime.

Each entry in the parts list defines one part to spawn:

- partPrefab: the specific prefab from the Switch Parts folder
- moduleIndex: written to the instantiated part's SwitchPart.moduleIndex
- localOffset: position relative to the spawn point (tablePose.position) in the table plane
- localRotation: initial rotation applied to the spawned part
- groupLabel: human-readable label for Inspector clarity only

Two separate assets exist:

- SwitchSpawnLayout-MR.asset: used by MR-SampleScene, references prefabs from MRTemplateAssets/Prefabs/Switch Parts
- SwitchSpawnLayout-AR.asset: used by AR-SampleScene, references prefabs from MobileARTemplateAssets/Prefabs/Switch Parts

---

SwitchSpawnerMR.cs (Assets/Scripts/MR Scripts)

Type: MonoBehaviour

Attached to: SwitchSpawner GameObject in MR-SampleScene

Purpose: Handles spawn triggering on Meta Quest 3, and moves snap zones and visual guides to the spawn position.

Input handling:

- Editor: left mouse click via Mouse.current.leftButton.wasPressedThisFrame
- Quest: right controller grip button via InputActionReference (XRI RightHand/Select)

Spawn attempt chain (same in both editor and Quest paths):

1. AR Foundation raycast against detected planes (ARRaycastManager.Raycast)
2. Physics raycast against plane MeshColliders from ARPlaneColliderGenerator
3. Fallback: fixed position in front of camera using fallbackSpawnDistance and fallbackYOffset

On successful spawn, moves snapZonesRoot and visualGuidesRoot to tablePose.position with tablePose.rotation, enables both, instantiates all 45 parts from the layout, overrides moduleIndex on each, enables AssemblyManager, notifies SpawnPreview to hide the placement ghost, and starts a 0.5-second coroutine to trigger AssemblyHintSystem.RefreshHints() after AssemblyManager has initialised.

Assembly orientation comment: SpawnAllParts contains two commented lines for snapZonesRoot.transform.rotation and visualGuidesRoot.transform.rotation. Using tablePose.rotation keeps snap zones and visual guides horizontal (floor-aligned, user-friendly). Using Quaternion.Euler(-90f, 0f, 0f) rotates both to vertical, aligning them with the assembly zone but making the visual guide perpendicular to the floor. See Section 11 (Limitations) for the full coordinate space mismatch explanation.

SwitchSpawnerAR.cs (Assets/Scripts/AR Scripts)

Type: MonoBehaviour

Attached to: SwitchSpawner GameObject in AR-SampleScene

Purpose: Mobile AR equivalent of SwitchSpawnerMR. Handles touchscreen tap input for Android and mouse click input in the editor. Uses the same three-step spawn raycast chain. Additionally calls LostPartRecoveryAR.InitFloor(tablePose.position) after spawning so the static floor is centered at the correct world position before any parts can fall through.

Input handling:

- Editor: left mouse click via Mouse.current.leftButton.wasPressedThisFrame
- Android: Touchscreen.current.primaryTouch.press.wasPressedThisFrame

No InputActionReference field — mobile touch is read directly from Unity's Input System. No controller-specific fallback path. All other spawn behaviour is identical to SwitchSpawnerMR, including the visualGuidesRoot movement, SpawnPreview notification, and delayed hint refresh coroutine.

Key difference from SwitchSpawnerMR: passes the full tablePose.position (Vector3) to LostPartRecoveryAR.InitFloor(), not just the Y component. This allows the static floor to be centered at the correct XZ position under the spawn area rather than at the world origin.

LostPartRecoveryAR.cs (Assets/Scripts/AR Scripts)

Type: MonoBehaviour

Attached to: AssemblyManager GameObject in AR-SampleScene only

Purpose: AR-specific replacement for AssemblyHintSystem.RecoverLostParts. Manages a permanent static physics floor collider and a two-frame kinematic teleport coroutine that allows recovered parts to land on the static floor reliably.

Why this exists separately: AssemblyHintSystem.RecoverLostParts uses FindFirstObjectByType<SwitchSpawnerMR> to locate the assembly origin, which returns null in AR-SampleScene where only SwitchSpawnerAR is present. AR-SampleScene also has a distinct floor instability problem specific to ARCore plane lifecycle (see Section 11) that requires a static floor solution not needed in MR-SampleScene.

Key methods:

InitFloor(Vector3 detectedPlanePose): called once by SwitchSpawnerAR immediately after parts are spawned. Creates a permanent BoxCollider (8m × 8m × 4cm) at the detected plane's world XZ position and 2cm below its Y position. The floor is placed on the SwitchPart physics layer (Layer 8) to guarantee collision with all switch parts regardless of the Default layer collision matrix state. Guards against duplicate initialisation with a null check on _staticFloor.

RecoverLostParts(): wired to the RecoverButton's OnClick event in the Inspector. Iterates all non-kinematic SwitchPart instances, identifies those further than fallThroughYThreshold (default −8cm) below floor Y or further than distanceThreshold (default 2m) from the snap zones origin, and starts TeleportAndRelease coroutine for each.

TeleportAndRelease(Rigidbody, Vector3) (private coroutine): sets isKinematic = true, moves the part to the target position above the floor, waits two WaitForFixedUpdate frames for the physics BVH to register the new position, then sets isKinematic = false and zeros velocity. The two-frame wait is critical — placing and releasing in the same frame allows parts to tunnel through the floor before the broadphase updates.

Inspector fields: snapZonesRoot (drag SnapZones-Transformed here), floorSize (default 8m), fallThroughYThreshold (default −0.08m), distanceThreshold (default 2m), recoveryHeight (default 0.20m), recoverySpread (default 0.05m random XZ offset to prevent exact stacking on recovery).

ARAdaptiveUI.cs (Assets/Scripts/AR Scripts)

Type: MonoBehaviour

Attached to: AssemblyHints GameObject in AR-SampleScene only

Purpose: Repositions and resizes the three UI elements (hint text, counter text, recover button) when the device orientation changes between portrait and landscape.

Why this exists: Screen Space — Overlay canvas elements are positioned in pixel coordinates. On Samsung A53 (2400×1080 portrait), elements positioned correctly in portrait appear outside the visible area in landscape (1080×2400) because the axis lengths invert. ARAdaptiveUI polls Screen.width and Screen.height every frame and calls ApplyLayout() when either dimension changes.

How it works: defines a fixed reference resolution (1080×2400 portrait baseline) matching the target device. Computes widthScale = Screen.width / ReferenceWidth and heightScale = Screen.height / ReferenceHeight independently. For each RectTransform, applies anchoredPosition scaled by the respective axis scale and sizeDelta using Mathf.Min(widthScale, heightScale) for both dimensions to preserve aspect ratio. The result keeps elements at proportionally consistent positions at both portrait and landscape edges regardless of resolution.

Inspector fields: hintTextRect, counterTextRect, recoverButtonRect — drag the corresponding RectTransform from the AssemblyHints Canvas children. Baseline position and size constants are defined as private static readonly Vector2 in the script and match the user's established layout (hintText at (-250, 900) size (1000×100); countText at (-250, 1000) size (1000×100); button at (0, -1000) size (400×160)).

AssemblyManager.cs

Type: MonoBehaviour (Singleton)

Attached to: AssemblyManager GameObject

Purpose: Central rules engine. Tracks which zones are satisfied, evaluates prerequisites, and determines when the full assembly is complete.

Key methods:

- EvaluateZone(zone): checks both AND prerequisites (all must be satisfied) and OR prerequisites (at least one must be satisfied). Calls zone.SetZoneActive(true/false) based on result
- OnZoneSatisfied(zoneId): called by AssemblySnapZone when a part snaps. Re-evaluates all unsatisfied zones (satisfying one zone may unlock others), checks assembly completion, refreshes hints
- OnPartUnsnapped(zoneId): called by AssemblySnapZone when a part is removed. Re-evaluates all zones including those that may have been locked by the removed zone's downstream dependencies. Refreshes hints and counter
- ResetAllZones(): two-pass reset — first clears all isSatisfied and disables all zones, then re-evaluates to restore Phase 0 zones to active

allZones list: serialised Inspector list of all 45 AssemblySnapZone components. Must be populated using **ABB Tools → Populate AssemblyManager Zone List** after any hierarchy change. Without this, no zones are known to the manager, and nothing activates.

AssemblySnapZone.cs

Type: MonoBehaviour

Attached to: all 45 snap zone GameObjects in SnapZones-Transformed
Purpose: The intelligence of each individual snap zone. Controls the XR Socket Interactor's active state, manages visual guide state transitions, validates snapping parts, and notifies AssemblyManager.

Key fields:

- zoneId: unique string identifier matching the zone's GameObject name exactly. Used by prerequisite lists
- acceptedPartType: which SwitchPartType this zone accepts
- acceptedModuleIndex: which module's part this zone accepts (only checked when ignoreModuleIndex=false)

- ignoreModuleIndex: when true, accepts any part of the correct type regardless of module. Currently false on all receiver module zones due to the interchangeability bug documented in Section 11
- visualGuide: reference to the corresponding VisualSnapGuide component, assigned automatically by VisualGuideGenerator
- prerequisiteZoneIds: AND list — all must be satisfied before this zone activates
- orPrerequisiteZoneIds: OR list — at least one must be satisfied (empty = always met)
- isOptional: when true, this zone does not count toward assembly completion

State machine:

- Inactive → (all prerequisites met) → Active/Ready → (correct part snapped) → Satisfied
- Satisfied → (part un-snapped) → Active/Ready

Hover behaviour: OnHoverEntered only responds to the correct part type (IsCorrectPart). Wrong-type zones remain silent, avoiding the red-flash bug where multiple zones competed to highlight the same part.

Un-snap handling: OnSelectExited checks isSatisfied before acting (prevents false triggers from the initial ejection of wrong parts). Restores Rigidbody to dynamic, resets zone state, updates visual guide.

PartTypeSelectFilter.cs

Type: MonoBehaviour implementing IXRSelectFilter

Attached to: all 45 snap zone GameObjects alongside AssemblySnapZone

Purpose: Physical gate on the XR Socket Interactor. If this filter returns false for a part, the socket physically cannot snap it — no hover ghost is shown and no snap event fires, regardless of AssemblySnapZone's state.

Must be referenced in the XR Socket Interactor's **Starting Select Filters** list in the Inspector. Without this wiring, the filter is not invoked, and any part can snap into any zone.

Fields: acceptedType, acceptedModuleIndex, ignoreModuleIndex — must match the corresponding AssemblySnapZone fields exactly.

VisualSnapGuide.cs

Type: MonoBehaviour

Attached to: each Guide_ GameObject inside the VisualGuides hierarchy

Purpose: Controls the appearance of one ghost mesh based on the corresponding snap zone's state.

States and colours:

- Inactive: renderer disabled (invisible)
- Ready: bright blue (0.2, 0.8, 1.0, 0.40) — zone is active and waiting
- HoverValid: bright green (0.0, 1.0, 0.4, 0.75) — correct part is hovering
- Satisfied: faint green (0.1, 1.0, 0.1, 0.20) — part successfully placed

The linkedZoneId field stores the corresponding zone's ID for debugging purposes. The actual link is the visualGuide reference on AssemblySnapZone, set by VisualGuideGenerator.

AssemblyHintSystem.cs

Type: MonoBehaviour

Attached to: AssemblyManager GameObject

Purpose: Gold-highlights all placeable parts, updates hint text with next required part types, updates the completion counter, and recovers lost parts.

RefreshHints(): called after every snap/unsnap and 0.5 seconds after spawn. Finds all active unsatisfied zones, finds all spawned non-snapped parts matching those zones (type + module), applies gold highlight to them, updates hint and counter text.

IsSnapped(): checks Rigidbody.isKinematic — snapped parts have isKinematic set to true by AssemblySnapZone.OnSelectEntered. Un-snapped parts are dynamic.

RecoverLostParts(): finds all non-snapped parts further than lostPartDistance (default 3m) from the SnapZonesRoot position. Teleports them back above the assembly origin with zeroed velocity. Wired to the RecoverButton's OnClick event.

PartLabelDisplay.cs

Type: MonoBehaviour

Attached to: AssemblyManager GameObject

Purpose: Shows a world-space TextMeshPro label above a part when the controller ray hovers over it.

ShowLabel(part, worldPosition): activates the PartLabel prefab instance, positions it above the part, rotates it to face the camera (LookRotation), and sets the TMP text to "{partType}\n{moduleName}". Called by SwitchPart.OnHoverEntered/OnHoverExited.

labelPrefab: a World Space Canvas containing one TextMeshPro child. Instantiated once on Start and reused by showing/hiding.

ARPlaneColliderGenerator.cs

Type: MonoBehaviour

Attached to: XR Origin GameObject

Purpose: AR Foundation's AR Plane Manager creates visual plane GameObjects, but they have no physics colliders by default. This script subscribes to the ARPlaneManager.planesChanged event and adds or updates a MeshCollider on each plane as it is added or refined. This enables Rigidbody parts to land on detected surfaces (table, floor) and enables the physics raycast fallback in the spawner to hit plane geometry.

Without this script, dropped parts fall through all detected surfaces and the spawner's physics raycast fallback produces incorrect spawn positions.

SpawnPreview.cs

Type: MonoBehaviour

Attached to: SpawnPreview GameObject

Purpose: Shows a transparent flat quad (SpawnPreviewPrefab) on the detected surface as the user aims the controller or phone camera at a plane, before spawning. When OnPartsSpawned() is called by the spawner, the preview is disabled permanently.

How it works: Every frame before spawn, raycasts from the screen center against AR-detected planes. On hit, it positions and orients the preview quad at the hit point, rotated 90 degrees from the plane normal so it lies flat on the surface. Preview scale (2m × 1.5m default) represents the approximate footprint of the full spawn layout.

Editor Scripts (Assets/Editor)

Editor scripts only compile and run inside the Unity Editor. They have no effect in device builds. All add options to the **ABB Tools** menu in the Unity menu bar.

AssemblyManagerSetup.cs

Menu: ABB Tools → Populate AssemblyManager Zone List

Finds all AssemblySnapZone components in the current scene using Object.FindObjectsByType<AssemblySnapZone>() and assigns them to AssemblyManager.allZones. Marks the AssemblyManager dirty so Unity saves the change.

When to run: after any change to the SnapZones hierarchy (adding, removing, or renaming zones). Must be run in MR-SampleScene and AR-SampleScene separately.

SnapZoneGhostSetup.cs

Menu: ABB Tools → Add Ghost Meshes to All Snap Zones

Adds a MeshFilter (Unity Cube mesh) and MeshRenderer (SnapZoneGhostMaterial) to every zone without an existing renderer. This was the original approach to visualising snap zones before the VisualGuideGenerator was added. These ghost cubes pile at (0,0,0) and are now always hidden by the updated AssemblySnapZone.cs (_zoneRenderer.enabled = false unconditionally). The tool still works for diagnostics.

SnapZonePositioner.cs

Menu: ABB Tools → Print Part World Centers

Reads all MeshRenderer components from the selected GameObject (select ABB_Switch root before running) and logs each child's name, bounds.center, and bounds.size to the Console.

Historical use: used to position snap zones at mesh centers before the (0,0,0) pivot architecture was understood. Now used diagnostically to verify visual guide placement matches expected part positions.

VisualGuideGenerator.cs

Menu:

- ABB Tools → Generate Visual Snap Guides (Name-Based)

- ABB Tools → Align Zone Triggers to Visual Guides
- ABB Tools → Clear Visual Snap Guides

Generate Visual Snap Guides (Name-Based):

Reads the ZoneToPartName dictionary mapping each zoneId to the corresponding part name in the ABB_Switch hierarchy. For each AssemblySnapZone, finds the matching MeshRenderer in ABB_Switch (must be enabled in scene), reads its bounds.center and bounds.size, creates a Guide_ GameObject at that world position with a Cube mesh and SnapZoneGhostMaterial, adds a VisualSnapGuide component, and assigns it to the zone's visualGuide field.

The dictionary maps 45 entries. If a name does not match, the guide is created at world (0,0,0), and a warning is logged. Correct the name in the dictionary and re-run.

Align Zone Triggers to Visual Guides:

For each AssemblySnapZone with a visualGuide assigned, it converts the guide's world position into the zone's local space using zone.transform.InverseTransformPoint(guideWorldPos) and sets this as the BoxCollider center. Also sets the collider size proportional to the guide's world-space scale with 25% tolerance. This is the critical step that makes triggers fire where the user sees the ghost mesh.

Run order: Generate first, then Align. If ABB_Switch reference is repositioned, regenerate and realign.

Clear Visual Snap Guides: destroys the VisualGuides root and unlinks all visualGuide references from zones.

---

8. Assets Reference

Switch Part Prefabs

Location: Assets/MRTemplateAssets/Prefabs/Switch Parts/ and Assets/MobileARTemplateAssets/Prefabs/Switch Parts/ (AR)

Each of the 45 prefabs has these four components:

1. **Rigidbody**: Mass=0.5, Linear Damping=5, Angular Damping=8, Collision Detection=Continuous Dynamic, Interpolate=Interpolate, Use Gravity=true
2. **Compound colliders** (Box/Capsule/Sphere): one or more primitive colliders approximating the part geometry. No Mesh Colliders on Rigidbodies (concavity instability). SwitchPartPhysics material applied to all

3. **XR Grab Interactable**: Movement Type=Kinematic, Throw On Detach=false, Force Gravity On Detach=true. Colliders list explicitly references the part's colliders
4. **SwitchPart.cs**: partType set to the corresponding SwitchPartType enum value. moduleIndex overridden at runtime by the spawner from the layout entry

Scale: MR prefabs at scale 0.2 (approximately true-to-life size when the parent ABB_Switch is at 0.002). AR prefabs at scale 0.1.

Physics Layer: all set to SwitchPart (Layer 8). Migrated to SwitchPartGrabbed (Layer 9) while grabbed.

SwitchSpawnLayout Assets

SwitchSpawnLayout-MR.asset and **SwitchSpawnLayout-AR.asset**: 45-entry lists defining spawn positions for each part. Offsets use 0.25m spacing across a 2.0m × 1.5m footprint (MR). AR layout uses offset values of 0.08m.

SnapZoneGhostMaterial.mat

URP/Lit material, Surface Type=Transparent, Base Color=(0.2, 0.6, 1.0, 0.3). Applied to all VisualGuide ghost meshes and to SpawnPreviewPrefab-AR, giving the AR spawn preview the same semi-transparent blue appearance as the snap zone guides.

Contact Windows.mat: URP/Lit opaque, transparent-grey tint approximating the physical contact window (translucent plastic housing) on the switch. Applied to all ContactWindow part prefabs. Also assigned as the SpawnPreviewPrefab-MR material, giving the MR spawn preview a light-grey floor ghost.

Fasteners and Clips.mat: URP/Lit opaque, yellow-gold colour matching ABB's hardware fastener and clip colour specification. Applied to ExternalFastener, MountingClip, InterlockingNut, and ModularRod prefabs across all modules.

Housings.mat: URP/Lit opaque, dark grey. Applied to all housing shell parts (BottomHousing, TopHousing, RightHousing, LeftHousing) and to WhiteStrip (the white strip uses the same base shader but its albedo value overrides the dark grey).

Lugs and Rings.mat: URP/Lit opaque, blue. Applied to TerminalLug and InterlockingDriveRing prefabs across all receiver modules.

Red Strip.mat: URP/Lit opaque, red with ABB embossed lettering approximated by colour. Applied to RedStrip in the Driver Module.

Top Covers.mat: URP/Lit opaque, black. Applied to TopCover in the Driver Module and ContactCover in all Receiver Modules.

SwitchPartPhysics.physicsMaterial

Dynamic Friction=0.6, Static Friction=0.6, Bounciness=0, Friction Combine=Maximum, Bounce Combine=Minimum. Applied to all colliders on all 45 part prefabs and to snap zone trigger colliders. The zero-bounciness setting eliminates the collision-explosion behaviour where parts bounced violently on contact.

PartLabel Prefab

World Space Canvas (scale 0.0003) with one TextMeshPro child (Width=300, Height=80, Font Size=60). Instantiated once by PartLabelDisplay and reused for all hover labels.

SpawnPreviewPrefab

A Quad with a semi-transparent material. Displayed on detected planes before spawn to show the layout footprint.

ABB_Switch.prefab

The assembled reference switch (scale 0.002, all children at their CAD-import positions). Kept disabled during gameplay. Enabled only when running editor tools (VisualGuideGenerator reads its mesh bounds). Must be placed at world position (0,0,0) when generating guides.

---

9. ABB Tools Editor Menu

Located in the Unity menu bar. These options only appear in the Editor and have no effect in device builds.

Menu Item	When to Use
Populate AssemblyManager Zone List	After any change to SnapZones hierarchy. Run in each scene separately.
Print Part World Centers	Diagnostic. Select ABB_Switch root first. Logs bounds.center of all 45 parts
Add Ghost Meshes to All Snap Zones	Legacy diagnostic tool. Adds cube meshes to zone GameObjects at (0,0,0). Now superseded by VisualGuideGenerator.
Generate Visual Snap Guides (Name-Based)	After enabling ABB_Switch at (0,0,0). Creates VisualGuides with correct world positions
Align Zone Triggers to Visual Guides	Always run after generating Visual Snap Guides. Maps Box Collider center/size to guide world positions

Menu Item	When to Use
Clear Visual Snap Guides	Destroys VisualGuides root and unlinks all zone references. Run before regenerating

Typical workflow after a hierarchy change:

1. Enable ABB_Switch, ensure it is at position (0,0,0)
2. ABB Tools → Clear Visual Snap Guides
3. ABB Tools → Generate Visual Snap Guides (Name-Based)
4. ABB Tools → Align Zone Triggers to Visual Guides
5. ABB Tools → Populate AssemblyManager Zone List
6. Disable ABB_Switch
7. Test in Play mode or build to device

---

10. Build Instructions

Building for Meta Quest 3

1. Open **MR-SampleScene**
2. **Edit → Project Settings → XR Plug-in Management → Android tab**: enable OpenXR with Meta Quest feature group, disable ARCore
3. **File → Build Settings**: Platform = Android, Run Device = your Quest 3 (must be connected via USB with USB Debugging enabled)
4. Add MR-SampleScene to Scenes In Build (index 0)
5. Click **Build and Run**
6. First build: 5–10 minutes. Subsequent incremental builds: 1–2 minutes
7. App installs and launches automatically on Quest. Access later from **Apps → Unknown Sources**

Testing without building (faster iteration): Connect Quest via USB, open Meta Horizon Link on PC, press Play in the Unity Editor. Streams PC rendering to Quest. Note: passthrough and plane detection may behave differently via Link compared to a standalone build. Always do a final standalone build for validation.

Building for Android Mobile

1. Open **AR-SampleScene**

2. **Edit → Project Settings → XR Plug-in Management → Android tab**: enable ARCore, disable OpenXR
3. Verify the Android device is ARCore-certified

   (check developers.google.com/ar/devices)
4. Enable USB Debugging on the phone (Settings → Developer Options)
5. **File → Build Settings**: Platform = Android, Run Device = your phone
6. Add AR-SampleScene to Scenes In Build
7. Click **Build and Run**

Switching between builds: The only changes required are in XR Plug-in Management (ARCore vs OpenXR) and the Scene in Build. All scripts, prefabs, and GameObjects are otherwise compatible with both platforms.

Testing in the Unity Editor

MR-SampleScene in Editor: Press Play. The editor uses XR Simulation (configured in the Desktop XR Plug-in Management tab). Left mouse click spawns parts (editor path in SwitchSpawnerMR). Parts cannot be grabbed with the mouse — editor testing is primarily for validating spawn logic, visual guide positions, and prerequisite activation (visible in AssemblyManager Inspector during Play mode).

AR-SampleScene in Editor: Press Play with XR Simulation enabled in the Desktop tab and build target set to Android. Left mouse click on the simulated floor spawns parts. Same limitations for grab interaction.

---

11. Known Issues and Limitations

Coordinate Space Mismatch (Assembly Zone vs Visual Guide Orientation)

Root cause: the ABB switch FBX was exported from Blender with all 45 parts having their mesh vertices encoded in a vertical coordinate frame (Y-axis = switch long axis). All child parts in the original import are rotated -89.98 degrees on X with scale 100 to compensate visually.

When XR Socket Interactor snaps a part, it moves the part's pivot (Transform.position, which is at world origin for all parts) to the socket position (also at world origin since SnapZones-Transformed is zeroed). The mesh then renders at pivot + vertex offsets. Because vertex offsets encode a vertical orientation, the assembled switch always appears vertical regardless of how ABB_Switch or SnapZonesRoot is rotated.

Visual guides were generated from ABB_Switch after rotating it -90 on X (horizontal). The guides are in the horizontal frame. The assembly zone is in the original vertical frame. These two frames are 90 degrees apart.

Current state: snap zones and visual guides are both horizontal (tablePose.rotation). Assembly appears vertical. Swapping to Quaternion.Euler(-90f, 0f, 0f) in SpawnAllParts rotates the snap zones vertical (matching the assembly zone) but makes the visual guides vertical too — both systems are always in the same frame, just not the assembly zone frame.

Permanent fix: re-export all 45 FBX parts from Blender using the following export settings in Blender and Import settings in Unity.

A. Export settings in Blender

I selected all 45 switch parts in the blender and set these settings:

1. Path Mode: 'Copy'. Also, select the folder icon on the right of this option to enable the setting for all selected parts (this setting is useful if your 3D Object has texture which you need to import as well along with the object)

2. Batch Mode: Off (default)

3. Include

    Limit to: Selected Objects

4. Object Types (default)

5. Transform:

    Scale = 1.00 (default)

    Apply Scalings = FBX All (important)

    Forward = -Y Forward (very important)

    Up = Z Up (default) Apply Unit (checked)

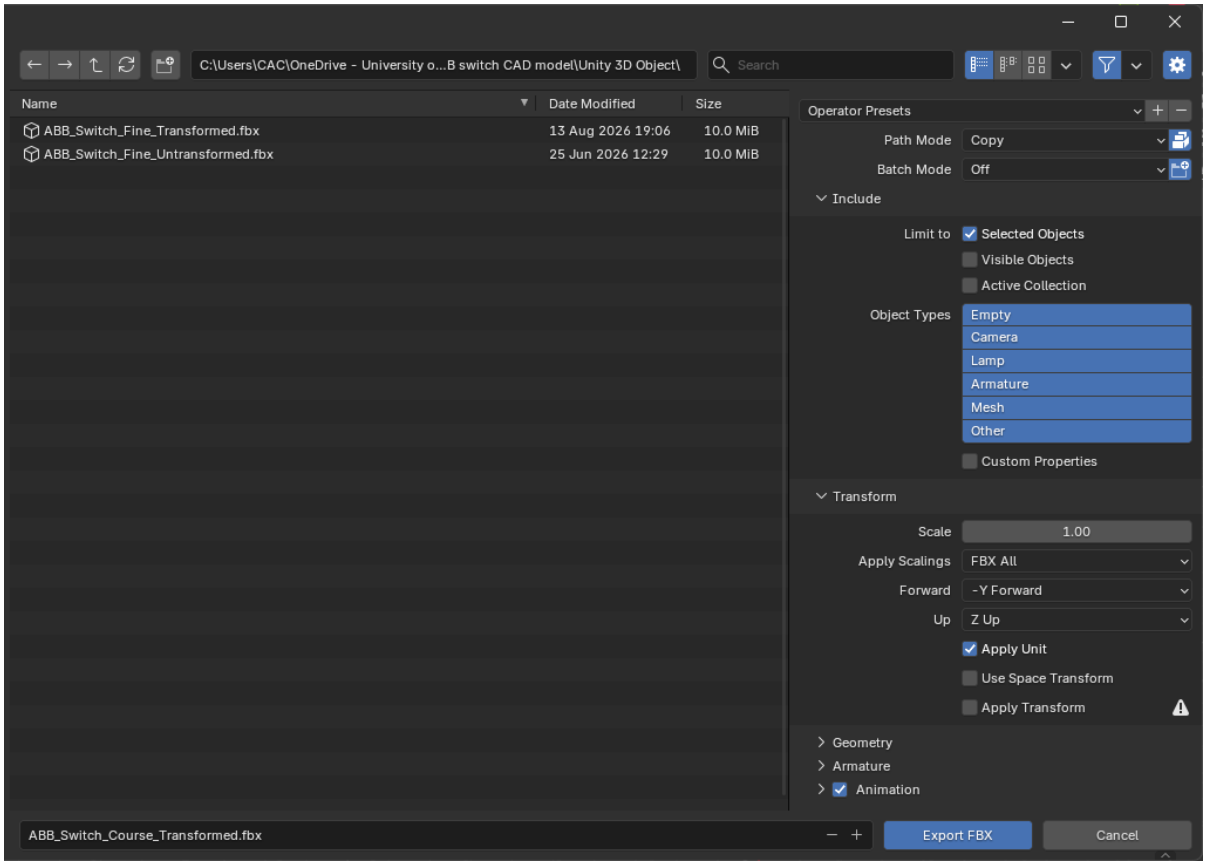
    Use Space Transform (unchecked)

    Apply Transform (unchecked)

6. Geometry (default)

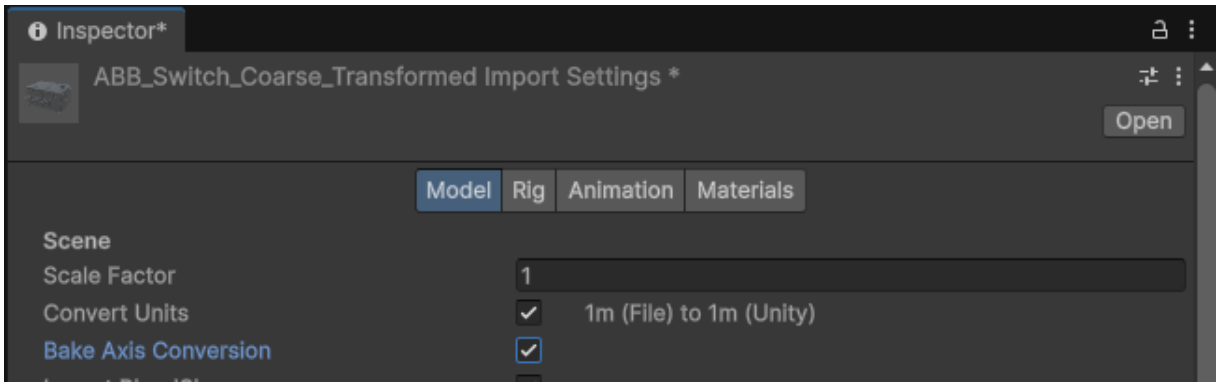
7. Armature (default)

8. Animation (default)



B. Import settings in Unity

When the exported FBX file is dragged or imported into Unity, select the asset (3D Object) and check the 'Bake Axis Conversion' checkbox in the Model menu in the Inspector View. Clicking the Apply button will set the scale to 1,1,1 and the rotation to 0,0,0 fixing the coordinate mismatch issue for once and for all.



This bakes the -90 X rotation into vertex positions as a horizontal orientation, making all three systems (snap zones, visual guides, assembly zone) compatible with a horizontal tablePose.rotation.

Alternative fix: implement a dual-socket proxy pattern. A Visual Socket at each guide's world position detects hover and user interaction. On hover, it finds the corresponding Assembly Socket (at world origin) and calls AssemblyManager.ForceSnap(zoneId, part),

which programmatically activates and selects the assembly socket. The part physically snaps to the assembly socket (vertical frame), but the user interacted at the visual socket position (horizontal frame). Estimated implementation: 2–3 days.

ignoreModuleIndex Interchangeability Bug

Root cause: with ignoreModuleIndex=true, a TerminalLug from Receiver-Module-1 can satisfy the snap zone of Receiver-Module-2. Because all part pivots are at world origin, any lug snaps to origin regardless of which zone triggered. The Module-2 zone reports isSatisfied=true, unlocking LeftHousing_2's prerequisites. But the physical assembly has the lug in Module-1's position. Downstream parts for Module-2 can then be assembled in mid-air.

Current workaround: ignoreModuleIndex=false on all receiver zones. Parts must match their designated module.

Permanent fix: same as above — Use the re-imported FBX with Apply Transforms so each part's pivot is at its geometric center and different module instances snap to different world positions.

Guardian Boundary Limits Movement

The Meta Quest Guardian boundary is a fixed OS-level safety system that cannot be expanded or removed via application code. Users stepping outside the boundary see the passthrough fade out. **Developer workaround**: disable Guardian entirely in Settings → Developer Mode (appropriate for supervised lab use only — not for production deployment).

Parts Spawning Direction Follows Camera, Not Raycast

The SpawnPreview and part spawn positions follow the camera's forward direction rather than the precise AR raycast hit point on the plane. This is because the spawner uses tablePose.position from the AR raycast for the SnapZonesRoot and VisualGuides positions, but parts are offset from this position using tablePose.rotation * entry.localOffset. The rotation is from the floor plane (horizontal), not the camera forward direction, so the layout spreads in a world-aligned grid rather than facing the user. In some viewing angles, this means parts spawn partially behind or to the side of the user.

Android Mobile Rendering Latency and Frame Rate Drops

The Android build exhibits noticeable frame rate drops and input latency that are not present in the Meta Quest 3 build. Three compounding causes:

1. **Mesh polygon density**: The switch parts were imported using the Fine SolidWorks STL export, which approximates curved surfaces to a very small angular tolerance. Fine export typically produces 50,000–200,000 triangles per complex part. With 45 parts simultaneously in the scene, the Samsung A53's Mali-G68 GPU processes 2–9 million triangles per frame as 45 separate draw calls (separate

materials prevent GPU batching). The Quest 3's §Snapdragon XR2 Gen 2 handles this comfortably; the A53's Exynos 1280 does not.

2. **Concurrent ARCore CPU load**: ARCore runs visual-inertial odometry, plane detection, and depth estimation concurrently with Unity's physics and render threads on the same CPU cores. The A53 shares its processor across all these tasks with no dedicated XR silicon. ARCore alone consumes 15–25% CPU on mid-range Android devices under active plane detection.
3. **Physics overhead**: Continuous Dynamic collision detection on 45 Rigidbodies with compound colliders runs near-$O(n^2)$ collision queries each fixed timestep, competing with ARCore on the same CPU threads.

Parts Gradually Sinking in AR Mobile

Switch parts slowly descend through the detected floor surface in the AR mobile build despite the static floor collider. Two contributing causes:

First, parts are spawned at tablePose.position.y + 0.05f — 5cm above the AR Foundation raycast hit — while the static floor is placed at tablePose.position.y - 0.02f. This creates a 7cm drop height. With Linear Damping = 5, a part takes 3–8 seconds to fall 7cm under gravity (heavily dampened settling, not fall-through). This is not falling through the floor but legitimate gravitational settlement to the floor surface.

Second, ARCore continuously refines plane estimates as the device moves, changing the detected floor Y. Parts that have settled at the original Y may appear to sink when ARCore adjusts the plane upward relative to them. The static floor does not update with ARCore, so it remains at the original spawn Y, but the AR plane visualizer (dotted mesh) moves to the new refined Y, creating a visual discrepancy that looks like sinking.

Hint Text Shows Generic Enum Names

AssemblyHintSystem displays partType.ToString() which maps to enum names like ExternalFastener rather than specific instance names like External-Fastener-Rcv3-L. Extending to specific names would require a lookup dictionary mapping (SwitchPartType, moduleIndex) → display name string.

---

12. Future Recommendations

Integrate Coarse FBX for Android Performance

Assets/Objects/ contains ABB_Switch_Coarse_Transformed.fbx — a correctly oriented (0,0,0 rotation, 1,1,1 scale) FBX export using the Coarse SolidWorks STL files. This model has significantly fewer triangles per part, making it suitable for the Android build's GPU budget.

To integrate:

1. Import ABB_Switch_Coarse_Transformed.fbx in Unity — all child meshes import with correct orientation
2. For each of the 45 prefabs in Assets/MobileARTemplateAssets/Prefabs/Switch Parts/, open the prefab in Prefab Edit mode, find the MeshFilter component, click the mesh picker, and select the corresponding mesh from the Coarse FBX. Names should match your existing naming convention
3. Adjust compound colliders if the Coarse geometry differs substantially from Fine (typically not necessary — colliders approximate shape, not exact surface)
4. Rebuild the AR-SampleScene and test frame rate

Similarly, ABB_Switch_Fine_Transformed.fbx contains correctly oriented Fine meshes for the MR build. Replacing current MR prefab meshes with these resolves the assembly zone perpendicularity issue (Section 11 Coordinate Space Mismatch) because the vertex positions encode a horizontal orientation from the correct Blender export, eliminating the need for any assembly correction rotation in SpawnAllParts.

Fix Gradual Sinking

Spawn Position Adjustment

Change + Vector3.up * 0.05f to + Vector3.up * 0.01f in SwitchSpawnerAR.SpawnAllParts. This reduces the spawn height above the static floor from 7cm to 3cm, cutting settlement time from 3–8 seconds to under one second. The deeper solution is to spawn parts with a small upward Rigidbody.AddForce impulse instead of a Y offset, then let damping settle them, avoiding any visible downward motion entirely.

Fix Latency/Performance Issues in Mobile App

LOD System for Mobile

Unity's LOD Group component automatically switches between mesh variants based on screen-space coverage. Assign the Fine mesh as LOD0 (close range) and the Coarse mesh as LOD1 (far range) for each part. Parts far from the camera render at reduced triangle count with no visual quality loss at viewing distance. This is the industry-standard solution for complex industrial AR scenes.

GPU Instancing for Repeated Parts

Parts sharing identical geometry (InterlockingNuts ×4, TerminalLugs ×6, ModularRods ×2, etc.) can be rendered in a single draw call using GPU instancing. Enable the GPU Instancing checkbox on the shared material assets. Unity automatically batches instances of the same mesh+material combination when Rigidbody.isKinematic = true (snapped parts become static and are batched). Reduces draw calls for repeated part types from N to 1 after assembly.

Static Batching for Assembled Parts

Once parts snap (isKinematic = true), they never move again and qualify for static batching. Call StaticBatchingUtility.Combine(assembledPartsArray, SnapZonesRoot) inside AssemblyManager.OnZoneSatisfied after every snap to progressively merge assembled geometry into single draw calls. This eliminates assembled part overhead from the per-frame draw call budget.

Specific Part Names in Hint Text

Add a Dictionary<(SwitchPartType, int), string> to AssemblyHintSystem mapping (type, moduleIndex) to a human-readable display name (e.g., (ExternalFastener, 3) → "External Fastener — Receiver Module 3"). Update RefreshHints() to use this dictionary instead of partType.ToString().

Assembly Experience

Dual-Socket Proxy Pattern: fully decouples the interaction frame (where the user drags parts) from the rendering frame (where assembled parts appear). Allows true interchangeability without the cascade bug. See Section 11 for implementation description.

Dynamic Snap Zone for ContactWindow: ContactWindow must be inserted into the bottom of ContactCover before ContactCover is placed. As a part of the workaround, the ContactCover snap zone is currently set as a pre-requisite of the ContactWindow zone. A dynamic implementation would: when ContactCover is picked up, activate the ContactWindow zone relative to the held ContactCover's position using a scene-space offset calculated from the ContactCover's real-world pose. This requires subscribing to XRGrabInteractable.selectEntered on ContactCover to reposition the ContactWindow zone dynamically.

Assembly Reference View: a World Space or screen-space canvas showing a rotating 3D render of the fully assembled switch as a reference image. Helps first-time users understand the target configuration. Can be rendered using a secondary camera pointing at a miniature assembled reference model placed off-scene.

Configurable Boundary Extension: implement a room-scale locomotion mode using the existing Locomotion component in XR Origin. Teleportation to different points around the switch allows users to assemble parts on the far side without physically walking outside the Guardian boundary.

Gamification and Modes

Timer and Leaderboard: add a float _assemblyStartTime recorded when parts spawn, stopped when AssemblyManager fires OnAssemblyComplete. Display elapsed time on

screen. Store top-3 times in a JSON file on the device. Prompt for name entry if the current session qualifies.

Tutorial Mode vs Test Mode: Tutorial Mode shows the hint system, gold highlights, part labels, visual guide, and a step-by-step coaching panel. Test Mode hides hint gold-highlighting, disables part labels (or shows names only on explicit button press), shows only the visual guide for feedback, and adds the timer. Toggle between modes from a main menu at app launch.

Assembly Completion Feedback: replace the Debug.Log in OnAssemblyComplete with a World Space canvas animation showing completion time, a restart button and a custom message. Add a particle effect or colour change on the fully assembled switch model.

Affordance Sound System: extend the existing snap sound system using Unity Audio Mixer. Separate audio buses for correct snap, incorrect attempt, assembly complete, and ambient AR environment sounds. Accessible from a global AudioManager singleton.

Spawn Ghost Preview Improvement: current preview is a static quad. Replace with a dynamic outline of all 45 part spawn positions (rendered as a point cloud or low-opacity bounding boxes matching each part's size and offset). Gives users an accurate preview of which parts will land where before committing to a spawn position.

Technical Improvements

ARCore Hand Tracking (select Android devices): Unity's XR Hands package combined with MediaPipe for Unity can provide hand tracking on Android phones that support ARCore's hand landmark extensions (Pixel 8 Pro, Galaxy S24 Ultra). Touch interaction would be replaced by pinch-to-grab on supported hardware.

Multiplayer Assembly: using Unity Netcode for GameObjects or Photon Fusion, multiple users in the same physical space could collaborate on assembly. Requires shared AR coordinate system (Meta Colocation Discovery for Quest, Google Cloud Anchors for Android).

Accuracy Scoring: beyond time tracking, implement spatial accuracy scoring. When a part is dropped near but not inside a snap zone, calculate the 3D distance and rotation error from the ideal snap pose. Average across all placements to produce an accuracy percentage independent of prerequisite enforcement. This measures fine motor precision rather than procedural knowledge.

QR Code or Image Tracking Based Anchor: instead of spawning at a detected plane, use ARFoundation's ARTrackedImageManager to spawn the assembly at a fixed QR code or printed marker placed on a physical workbench. This ensures the virtual assembly always aligns with the same real-world location session after session, without requiring spatial anchor persistence.